

RESTful APIs in Spring Boot expose CRUD operations over HTTP using endpoints like `GET /users`, `POST /users`, `PUT /users/{id}`, and `DELETE /users/{id}`, with JSON payloads and proper status codes (200, 201, 404). They leverage Spring MVC annotations to map requests to service methods, enabling stateless, scalable backends for frontend apps or microservices.

@RequestMapping maps HTTP requests to handler methods—e.g., `@RequestMapping(value = "/users", method = RequestMethod.GET)` handles GET calls. It's often replaced by shortcuts like `@GetMapping`, `@PostMapping` for brevity, and supports path variables (`/users/{id}`), request params, headers, and produces/consumes media types.

answer in bit long

RESTful APIs in Spring Boot are used to build lightweight, stateless web services that communicate over HTTP using standard methods (GET, POST, PUT, DELETE, PATCH) and resource-based URLs. They accept and return data typically in JSON or XML format, making them ideal for modern frontend frameworks (React, Angular) and mobile apps. For example, a `GET /api/products` fetches all products, `POST /api/products` creates a new product, `PUT /api/products/101` updates product with ID 101, and `DELETE /api/products/101` deletes it. Spring Boot auto-configures embedded servers (Tomcat), Jackson for JSON serialization, and exception handling (like `@ControllerAdvice`) to return meaningful HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error). These APIs are often secured with Spring Security, documented using Swagger/OpenAPI, and consumed by clients via `RestTemplate` or `WebClient`. They enable microservices communication, separation of frontend/backend concerns, and easy integration with third-party systems.

@RequestMapping is a core Spring MVC annotation used to map web requests to specific controller handler methods. It can be applied at the class level (to set a base URL prefix like `/api/users`) and at the method level (to define the exact endpoint). It allows configuration of:

- **HTTP method** (`method = RequestMethod.POST`),
- **URL patterns** (`value =("/{id})"` with path variables like `@PathVariable Long id`),
- **Request parameters** (`params = "active=true"`),
- **Headers** (`headers = "Content-Type=application/json"`),
- **Consumes/produces** media types (`consumes = MediaType.APPLICATION_JSON_VALUE`, `produces = MediaType.APPLICATION_JSON_VALUE`).

For convenience, Spring provides specialized shortcuts: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping` – which are simply composed annotations with the method pre-defined.

Example: `@GetMapping("/{id})"` is equivalent to `@RequestMapping(value =("/{id})", method = RequestMethod.GET)`. This annotation is fundamental for defining routing, enabling

flexible REST endpoint design, and supports RESTful principles like content negotiation and clean URL hierarchies.